GammaLex project descriptions

Ethics of AI for Autonomous Systems / Design of Human-Centered AI for Autonomous Systems · Fall 2026 – Spring 2027 · Dr. Stephanie Moore and Dr. Meeko Oishi

Three open-source projects in priority order, plus an optional fourth in the appendix. Each suits a mixed team of technical and non-technical students. **If only two teams are available, we ask for Projects 1 and 2.**

	Project 1	Project 2	Project 3	Project 4 (optional)
Title	Benchmarking context sufficiency and human oversight	Open-weight models for evidence extraction	Human-approved requirements models for new domains	OpenContextually beyond Python
Core result	500 reviewed cases, model comparison, oversight study	One open extraction model and an open evaluator	One approved requirements model, plus transfer to a second domain	Multi-language support, held-out benchmark, injection safety
Human factors	Trust calibration, situation awareness	Privacy, explainability, fairness	Accountability, automation bias	Trust, workload, safety
Strong fit	CS, statistics, OILS, health, law, policy	CS, ECE, linguistics, ethics, GLNS	Policy, social work, physics, CS	CS, ECE, security

The question behind all four projects

GammaLex builds context infrastructure for AI agents. Before an agent acts, is its context enough: present, current, consistent and permitted? All projects use one definition:

- Each **mandatory requirement** is **established**, **not established** (the check completed and failed: missing, stale, conflicting, restricted or ambiguous), or **undetermined** (the check could not complete, for example a source was unreachable).
- The outcome, in order: **insufficient** if any mandatory requirement is not established; otherwise **incomplete** if any is undetermined; otherwise **sufficient**.
- Example: two offices disagree on household size, so the outcome is insufficient. The income source is down, so it is incomplete.
- The check is deterministic (same records, requirements and time give the same result) and never asks a language model whether context is enough.

Common terms

- **Open source, for learning, no NDA.** Public repositories under github.com/gammalex-ai (code Apache-2.0, OpenContextually MIT, data CC BY 4.0). Students keep their copyright (DCO sign-off), are credited, and present what they built and learned at each review.
- **Public or synthetic source data only.** Participant data (reviews, study responses) is handled under a UNM IRB-approved protocol and published only with consent. No human-subjects work before an IRB determination; file in the fall.

- **Nothing from GammaLex's private code.** Teams define their own formats and build their own tools in the open.
- **GammaLex provides mentoring only:** a 30-minute check-in every two weeks, review of designs and pull requests, and replies within 2 business days. Contact: yen.kha@gammalex.com; GitHub issues for technical questions.
- **Teams secure their own compute and API access** (for example UNM CARC, free tiers, academic credits). Every deliverable must work with open-weight models alone.
- **Engineering:** Python 3.12, tests and CI on every repository; every reported number regenerated by a committed script.

Quick links

Area	Links
GammaLex	gammalex.com · Research · OpenContextually
Public data	DailyMed · ClinicalTrials.gov API · eCFR · Federal Register · CourtListener · Retraction Watch (Crossref) · Synthea
NM housing	Albuquerque Housing Authority · HUD income limits · NM SNAP rules (8.139 NMAC) · Census ACS
Quantum	Qiskit device snapshots · Qiskit Experiments
UNM	CARC · IRB

Project 1

Project title

Benchmarking context sufficiency and human oversight across domains

Project sponsor

GammaLex AI Inc. (Albuquerque, NM)

Point of contact

Yen Kha, Founder · yen.kha@gammalex.com · see Common terms for how we work together

Project overview and motivation

An agent can fail with no reasoning error at all: it acts on a superseded record, a value two systems disagree on, or a document it should not use. Most benchmarks score whether a model reaches the right answer; few score whether it recognizes it should not act yet. This project builds a large public benchmark of that skill, compares many models under identical conditions, and studies how sufficiency information should be shown to the people who supervise agents.

Key elements of the approach

- **Cases:** 500 independently reviewed cases in 2–3 domains (stretch: 1,000), from public records such as drug labels, trial records, regulations, retractions and court opinions.
- **Labels:** written and committed before any model runs; two annotators per case; revise the guide if agreement (κ) falls below 0.70 on a 60-case pilot.
- **Stress tests:** thousands of generated variations (evidence removed, values conflicting, dates moved) and growing evidence volume, reported separately from reviewed cases.
- **Models:** at least 6 (at least 4 open-weight), each tested with records only and with records plus written requirements; 5 samples per case; confidence intervals.
- **Fairness of the test:** splits by source document; the test split stays private until evaluation, with its hash published first.
- **Oversight study (after IRB):** people supervise a simulated agent shown (1) its answer, (2) plus an outcome, or (3) plus reasons and evidence. The outcome comes from a checker with a set error rate, so over-reliance can be measured.

Skills and engineering constraints

	Details
Team (4–6)	Python and data; statistics; domain readers (health, law, policy); an HCI or OILS lead for the study
Required skills	Python (pandas, requests), git and GitHub pull requests, statistics (confidence intervals, paired tests, Cohen's kappa), careful reading of source documents
Helpful skills	Experimental design and power analysis, Qualtrics, a web framework (for example React) for the study interface, running open-weight models with vLLM
Engineering constraints	Labels committed before any model runs. Records capped at 8,000 characters in the short-input condition. Strict answer parsing, at most 3 logged retries. Every number reproducible from saved raw responses. A cost cap on every run. Source licenses checked before redistribution.

Human-centered design considerations

- **Trust calibration:** measure over- and under-reliance separately.
- **Situation awareness:** does showing what changed, conflicts or is missing help supervisors?
- **Safety:** report how often each model calls insufficient context sufficient.
- **Fairness:** results per domain.

Major deliverables and desired outcomes

Deliverable	Done when (targets are proposals)
Reviewed benchmark	500 cases, at least 150 per domain; each failure type at least 15% of a domain's not-sufficient cases; 30–40% positive controls; every record cited with URL, retrieval date and SHA-256
Stress suite	At least 3,000 generated variations across 5 change types (evidence removed, conflicting value, moved date, restriction added, combinations); 5% spot-checked by a person
Volume tests	1, 10, 50 and 100+ records per case; documents up to 50,000 characters
Harness	pip-installable; one command reruns everything; at least 85% test coverage; identical results from saved raw responses
Results report	At least 6 models (4+ open-weight, in two size classes: up to 8B and 30B+ parameters) in both conditions. Primary metric: share of not-sufficient cases called sufficient, with Wilson 95% intervals; McNemar tests between conditions
Oversight study	IRB-approved; pilot with about 10 people; main study with at least 36 (12 per display)
Paper and leaderboard	Submission-ready paper; public leaderboard; test split published after evaluation with a Zenodo DOI

Milestones (proposed)

When	Milestone
Fall, weeks 1–4	Sociotechnical analysis and literature survey; choose domains; annotation guide v1
Fall, weeks 5–10	60-case pilot per domain and agreement measured; harness skeleton; IRB application filed
Fall, weeks 11–15	200 reviewed cases; first 3 models run; metrics and statistics plan frozen
Spring, weeks 1–8	500 cases; stress suite and volume tests; all models in both conditions; study pilot
Spring, weeks 9–16	Main study; analysis; paper, leaderboard and public release

Support that the students will receive

Mentoring as described in Common terms. Plus review of the case design, statistics plan and IRB protocol.

Recommended background reading

- Lee & See (2004). Trust in automation. doi.org/10.1518/hfes.46.1.50_30392
- Endsley (1995). Situation awareness in dynamic systems. doi.org/10.1518/001872095779049543

- Parasuraman, Sheridan & Wickens (2000). Levels of automation. doi.org/10.1109/3468.844354
- Liu et al. (2024). Lost in the middle. arxiv.org/abs/2307.03172
- Gebru et al. (2021). Datasheets for datasets. doi.org/10.1145/3458723

Stakeholder evaluation

- UNM faculty and graduate students (pharmacy, law, public administration) review case validity.
- Analysts, clinicians in training and paralegals take part in the study; UNM students pilot it.

(Optional) Notes on the design process elements

Fix the metrics and statistics plan before data collection (technical framing). Publish results whatever they show (technical evaluation).

Project 2

Project title

Open-weight models for evidence extraction

Project sponsor

GammaLex AI Inc. (Albuquerque, NM)

Point of contact

Yen Kha, Founder · yen.kha@gammalex.com · see Common terms for how we work together

Project overview and motivation

To check context, a system must first read its records: turn a drug label or a benefits letter into facts. Hand-written patterns are brittle; sending records to commercial models exposes sensitive data, changes without notice and invents values. This project asks whether a small open-weight model, run locally, can extract facts that each quote their exact source, well enough to keep sufficiency outcomes correct. The model only proposes; the deterministic rule decides.

Key elements of the approach

- **Task:** extract claims (entity, attribute, value, unit, date), each with the exact quoted text it came from.
- **Baselines first:** a rule-based extractor the team writes, an untuned open-weight model, and a larger open-weight model (plus a commercial model if the team has access).
- **Train only where needed:** fine-tune (for example LoRA) on the weaknesses the baselines reveal. No pretraining.
- **Checks:** the quote must match the record exactly and values must follow from it; added checks catch a wrong entity, negation, unit or date. Discarded claims count as misses.
- **Does it matter?** An open evaluator applies the outcome rule to public scenarios and measures how extraction errors change outcomes, especially false sufficient results.
- **Data:** a pilot of about 500 human-reviewed records in 2 domains, including some Spanish documents; expand toward 3,000 only if it helps.

Skills and engineering constraints

	Details
Team (4–6)	2–3 machine-learning builders, an annotation lead, an ethics or policy lead
Required skills	Python, PyTorch, Hugging Face Transformers and Datasets, Linux and Slurm job scripts (for CARC), JSON Schema, evaluation metrics (precision, recall, F1)
Helpful skills	PEFT and LoRA, constrained decoding to a JSON schema, vLLM or llama.cpp, 4-bit quantization, an annotation tool such as Label Studio
Engineering constraints	The model never judges sufficiency; it only proposes claims. No network at inference. Greedy decoding with pinned weights and seeds, and every run logged so it can be replayed. Runs over 50 GPU-hours need a written plan. Training and test data never overlap. Weights released only if the base model's license allows.

Human-centered design considerations

- **Privacy:** records stay on the user's machine.
- **Explainability:** every fact points to its quote; test whether reviewers check faster.
- **Safety:** measure invented values and obedience to instructions hidden in records.
- **Fairness:** results per domain and language, including Spanish.

Major deliverables and desired outcomes

Deliverable	Done when (targets are proposals)
Pilot dataset	500 records in 2 domains, all human-reviewed; 100 reviewed twice to measure agreement; at least 50 Spanish-language records; split 70/15/15 by source document
Baseline report (fall)	Rule-based, untuned open-weight and larger open-weight baselines compared on per-field F1, invented-value rate and discarded-claim rate
Extraction model	Open weights (up to 8B parameters) on Hugging Face with a model card; runs on one 24 GB GPU at 20+ records per minute (4,000-token records)
Checker and evaluator	At least 30 public scenarios; with human-verified claims, outcomes match hand-derived results on every scenario
Results	Exact-quote F1, invented-value rate, discarded-claim rate, outcome changes (especially false sufficient), cost and speed, with 95% intervals
Stretch	A second model size; a version that runs on a 16 GB laptop CPU

Milestones (proposed)

When	Milestone
Fall, weeks 1–4	Literature survey; claim schema v1 and labeling guide; compute secured (CARC request filed)
Fall, weeks 5–10	Label the 500-record pilot; write the rule-based baseline; run the open-weight baselines
Fall, weeks 11–15	Baseline report; acceptance thresholds frozen; training plan reviewed
Spring, weeks 1–8	Fine-tune on measured weaknesses; build checker, evaluator and scenarios
Spring, weeks 9–16	Outcome results; reviewer check; release model, data and code; paper

Support that the students will receive

Mentoring as described in Common terms. Plus review of the baseline report and training plan. The team secures its own GPU time (planning figure: up to 2,000 GPU-hours).

Recommended background reading

- Hu et al. (2021). LoRA. arxiv.org/abs/2106.09685
- Ji et al. (2023). Hallucination in natural language generation. arxiv.org/abs/2202.03629
- Greshake et al. (2023). Indirect prompt injection. arxiv.org/abs/2302.12173
- Mitchell et al. (2019). Model cards. arxiv.org/abs/1810.03993
- NIST (2023). AI Risk Management Framework. nist.gov/itl/ai-risk-management-framework

Stakeholder evaluation

- People who read these records professionally (lab staff, clinicians or caseworkers in training) judge whether cited facts are correct and quick to verify.
- UNM security or GLNS faculty review the privacy analysis.

(Optional) Notes on the design process elements

Set acceptance thresholds after the baseline report (technical framing). Released weights must respect the base model's license (ethical constraints).

Project 3

Project title

Human-approved requirements models for new domains

Project sponsor

GammaLex AI Inc. (Albuquerque, NM)

Point of contact

Yen Kha, Founder · yen.kha@gammalex.com · see Common terms for how we work together

Project overview and motivation

A context check can only test what it has been told matters: which facts are needed, from which source, and how recent. Experts write these requirements by hand from regulations and procedures, which is slow and a barrier to new domains. This project asks whether a language model can draft them from public documents, each citing its source passage, so an expert can review and approve them quickly, and whether experts then rubber-stamp bad drafts.

Key elements of the approach

- **Primary domain: New Mexico housing.** What must be established before a family with rental assistance is matched to a unit. Rules are real and public; households are synthetic.
- **Drafting:** a model proposes requirements in the team's own open format, each citing a passage. No citation, no requirement.
- **Review tool:** each requirement beside its passage, with accept, edit and reject actions, all logged.
- **Expert study (after IRB):** experts write requirements from scratch, or review a draft with planted errors. Measure time, missed and wrong requirements, and workload.
- **Rule change:** when a new year's income limits are published, measure how quickly and correctly the approved model is updated.
- **Spring transfer to quantum computing,** using only public Qiskit device snapshots and procedures. Check that a simulation used a declared device snapshot and met the study's stated requirements.

Housing data	Source
Federal rules	Voucher program, 24 CFR 982 and Part 5 (eCFR); HUD Voucher Guidebook
New Mexico figures	HUD income limits and Fair Market Rents by county and year
Local and state policy	Albuquerque Housing Authority; NM SNAP rules (8.139 NMAC); Housing New Mexico
Households	Synthetic, calibrated to public Census ACS figures; no real family data

Skills and engineering constraints

	Details
Team (4–6)	A policy, social work or law reader; Python and LLM skills; an HCI or OILS lead; a physics student for the spring transfer
Required skills	Python, running an LLM locally or by API, JSON Schema or YAML schema design, reading federal regulations (CFR) and state rules (NMAC)
Helpful skills	A web framework (for example React) for the review tool, accessibility testing (WCAG 2.2 AA), synthetic data generation, Qiskit basics
Engineering constraints	No requirement without a cited passage. Drafts must pass the team's schema before human review. Every draft, edit and approval logged. Synthetic households only. Each source document recorded with its retrieval date and hash.

Human-centered design considerations

- **Accountability:** who answers for a model-drafted, human-approved rule?
- **Automation bias:** planted errors measure rubber-stamping; the tool shows the source passage first.
- **Fairness:** a wrong rule falls on families; the team reviews the model for unequal effects.
- **Workload:** less expert time without lower quality.

Major deliverables and desired outcomes

Deliverable	Done when (targets are proposals)
Scope	The unit-match step of the Housing Choice Voucher program: household and income definitions (24 CFR Part 5), reasonable rent (24 CFR 982.507), housing quality standards (24 CFR 982.401) and local policy in the housing authority's Administrative Plan (24 CFR 982.54)
Approved housing model	Every requirement cited to a section and approved by an expert; tested on at least 50 synthetic households, each with 0–3 planted problems
Drafting pipeline	Open source; at least 95% of drafts pass the team's schema on the first try
Review tool	Requirement beside its passage; accept, edit and reject; every action logged; meets WCAG 2.2 AA
Expert study	Up to 8 caseworkers in 60–90-minute sessions, half writing from scratch and half reviewing a draft in which about 10% of requirements are planted errors; time, misses, wrong approvals and NASA-TLX recorded
Rule change	Next year's HUD income limits applied; update time and correctness measured
Quantum transfer	Device snapshots from at least 2 Qiskit releases; each requirement labeled as from a published procedure or a study assumption

Milestones (proposed)

When	Milestone
Fall, weeks 1–4	Map the housing pathway; collect source documents with retrieval dates and hashes; requirements format v1
Fall, weeks 5–10	Gold-standard model written; synthetic household generator; IRB application filed
Fall, weeks 11–15	Drafting pipeline v1; review tool prototype; pilot with 1–2 reviewers
Spring, weeks 1–8	Expert sessions; rule-change exercise
Spring, weeks 9–16	Quantum transfer; analysis; release and paper

Support that the students will receive

Mentoring as described in Common terms. Plus review of the requirements format and the study protocol.

Recommended background reading

- Parasuraman & Manzey (2010). Complacency and bias in automation use. doi.org/10.1177/0018720810376055
- Eubanks (2018). Automating Inequality. St. Martin's Press.
- Citron (2008). Technological due process. openscholarship.wustl.edu

- Amershi et al. (2019). Guidelines for human-AI interaction. doi.org/10.1145/3290605.3300233
- GammaLex: housing research

Stakeholder evaluation

- New Mexico caseworkers or housing navigators, recruited under IRB (up to 8 sessions).
- UNM quantum researchers review the spring transfer.

(Optional) Notes on the design process elements

Start the sociotechnical analysis with the housing pathway: family, housing authority, benefits office, landlord. Label each quantum requirement as from a published procedure or a study assumption.

Appendix: optional Project 4

Project title

OpenContextually beyond Python: safe, multi-language context for coding agents

Project sponsor

GammaLex AI Inc. (Albuquerque, NM)

Point of contact

Yen Kha, Founder · yen.kha@gammalex.com · see Common terms for how we work together

Project overview and motivation

Coding agents decide what to read before they change code, and they can read the wrong files or files with planted instructions. OpenContextually is an open-source tool that returns a small, ranked, explained set of files for a task, with no model and no network. Today it follows imports only in Python, and on its own benchmark it finds 40 of 48 expected files (83%). This project extends it to more languages, larger repositories and untrusted content.

Key elements of the approach

- **Languages:** import resolution for TypeScript/JavaScript, Go and Rust, as optional add-ons.
- **Benchmark v2:** 20+ new repositories, with a held-out set frozen before any tuning.
- **Scale:** usable on repositories of 100,000+ files, with an opt-in cache.
- **Safety:** flag instructions hidden in repository content (never silently edit them), and harden secret redaction.
- **Impact:** compare a coding agent with and without the tool on public bug-fix tasks.

Skills and engineering constraints

	Details
Team (4–6)	Python and parsers, security, an HCI lead for a developer study, a benchmark case writer
Required skills	Python 3.10+, tree-sitter grammars and queries, abstract syntax trees, pytest, profiling (cProfile), git
Helpful skills	TypeScript, Go or Rust; prompt injection and secret handling; the VS Code extension API
Engineering constraints	No model in file selection and no network. Byte-identical output for the same repository, task and version. Nothing written to disk unless the user opts in. New languages ship as optional add-ons. Every check deterministic, with its false-positive rate reported. Held-out repositories frozen before any tuning.

Human-centered design considerations

- **Trust:** do the one-line reasons make developers more accurate, or just more confident?
- **Safety:** hidden instructions must be visible to both agent and human.
- **Workload:** smaller packages should mean less review effort.

Major deliverables and desired outcomes

Deliverable	Done when (targets are proposals)
Language support	TypeScript/JavaScript, Go and Rust import resolution through tree-sitter grammars, as optional add-ons; at least 5 benchmark repositories per language
Benchmark v2	At least 20 new repositories; at least 8 held out and frozen before any tuning; file recall and excerpt recall reported per cohort, never combined
Scale	A 100,000-file repository in 120 seconds or less on first run, and 15 seconds or less with the cache, on a recent laptop (targets confirmed in the fall)
Safety check	At least 100 planted injections; at least 90% flagged; no more than 1 false flag per 10 clean repositories
Agent study	50 public bug-fix tasks run with and without the tool; success rate, tokens and files read

Milestones (proposed)

When	Milestone
Fall, weeks 1–4	Run the tool on familiar repositories; read the codebase; choose languages; benchmark guide
Fall, weeks 5–10	TypeScript/JavaScript resolver; 10 new repositories; held-out set frozen
Fall, weeks 11–15	Go resolver; injection corpus; scale profiling
Spring, weeks 1–8	Rust resolver; cache; safety check; developer study pilot
Spring, weeks 9–16	Agent study; one held-out evaluation; release

Support that the students will receive

Mentoring as described in Common terms. GammaLex adds plug-in points for languages and checks before the course starts. The tool keeps its rules: no model, no network, same output every run.

Recommended background reading

- OpenContextually: where to start and benchmark
- Jimenez et al. (2024). SWE-bench. arxiv.org/abs/2310.06770
- Greshake et al. (2023). Indirect prompt injection. arxiv.org/abs/2302.12173
- Model Context Protocol and tree-sitter

Stakeholder evaluation

- Developers who use coding agents (UNM students and staff, under IRB), plus OpenContextually users on GitHub.

(Optional) Notes on the design process elements

Treat repository content as untrusted input in the ethical design. Report held-out results honestly, including regressions.